

Part 3.

청킹 — 문서를 검색 단위로 자르기

청킹은 긴 문서를 검색에 적합한 작은 조각으로 나누는 작업이다. 임베딩 모델과 LLM은 한 번에 처리할 수 있는 길이가 제한되어 있고(컨텍스트 윈도우), 검색은 질문과 관련된 부분만 정확히 끌어와야 하므로 문서 전체를 그대로 넣을 수 없다.

왜 필요한가

- **토큰 한도:** 임베딩 모델은 보통 수백~수천 토큰까지만 한 번에 인코딩한다.
- **검색 정확도:** 너무 큰 단위로 자르면 무관한 내용이 섞이고, 너무 작으면 맥락이 끊긴다.
- **비용과 속도:** LLM에 넘기는 컨텍스트가 짧을수록 응답이 빠르고 토큰 비용도 줄어든다.

어떻게 자르는가

- **고정 길이 분할:** 글자 수나 토큰 수 기준으로 일정하게 자른다. 구현이 가장 간단하다.
- **오버랩:** 이웃한 청크가 일부 겹치도록 잘라 문장 중간에서 맥락이 끊기는 문제를 줄인다.
- **구조 기반 분할:** 문단·제목·섹션 같은 자연스러운 경계를 따라 잘라 의미 단위를 보존한다.
- **권장 크기:** 보통 300~800 토큰, 오버랩은 10~20% 수준에서 시작해 데이터에 맞게 조정한다.

청킹이 왜 중요한가?

RAG의 검색 정확도는 청크 크기에 크게 좌우된다. 청킹은 단순히 자르는 작업이 아니라, 검색이 잘 되도록 의미 단위를 설계하는 일이다.

청크가 너무 길면

- 검색이 매우 세밀해져서 질문과 정확히 맞는 부분을 찾아낼 확률이 높다.
- 그러나 맥락이 잘려 나가 "의미 단절"이 발생할 수 있다.
- 한 청크 안에 여러 주제가 섞여 임베딩 벡터가 모호해지고 검색 정확도가 떨어진다.

청크가 너무 짧으면

- 문맥 보존에 유리하고 전체 흐름을 함께 제공 가능하다.
- 그러나 불필요한 정보까지 검색 결과에 포함되어 정확도가 떨어진다.
- 문장이 중간에서 끊기고 맥락이 사라져, LLM이 답변에 활용할 정보가 부족해진다.

경험적으로 한국어 문서는 ~~200500자~~, ~~오버랩 1020%~~ 정도가 출발점으로 무난하다.

| "정확한 검색은 좋은 청킹에서 시작된다."

청킹 전략 비교 — 4가지 방식

- **Fixed-size (고정 길이)**: 정해진 토큰·글자 수로 자른다. 가장 단순하며 보통 앞뒤가 겹치도록 오버랩(sliding window)을 둔다.
- **Recursive (재귀적 분할)**: 문단·줄바꿈·문장 같은 구분자 우선순위를 따라 큰 단위부터 자르고, 한도를 넘으면 더 작은 단위로 다시 자른다. RAG의 사실상 표준 baseline이다.
- **Document-based (구조 기반)**: Markdown 헤더, HTML 태그, 코드 블록 등 문서 고유 구조를 경계로 삼는다. 구조가 분명한 문서일수록 효과가 크다.
- **Semantic (의미 기반)**: 문장별 임베딩 유사도가 크게 떨어지는 지점을 의미 경계로 보고 자른다. 품질은 가장 좋지만 인덱싱 비용이 든다.

고정 길이 + 오버랩 청킹 실습

1. `doc.txt` 파일을 LMS에서 다운받는다.
2. 코드를 작성하고 실행한다.

250자씩 전진하면서 300자를 떼어낸다 → 인접 청크끼리 50자가 겹친다 → 문장 중간에서 잘려도 옆 청크에 그 맥락이 남는다.

코드 해석: `doc.txt` 를 UTF-8로 통째로 읽어 `text` 에 담고, `chunk_text_fixed_overlap` 함수에 넣어 300자씩, 50자 오버랩으로 자른다. 이전 청크의 50자와 다음 청크의 50자가 겹친다.

오버랩은 왜 필요한가?

고정 길이 청킹은 의미 단위를 무시하고 글자 수만 보고 자르기 때문에, 청크 경계가 문장 한가운데 떨어지는 일이 흔하다. 오버랩은 인접 청크가 일부 겹치도록 만들어 그 경계에서 끊긴 맥락을 옆 청크가 받아주게 한다.

오버랩이 없으면

- **문장 절단**: 경계가 "환불은 7일 " / "이내에만 가능하다"처럼 한 문장을 둘로 쪼갬다.
- **검색 실패**: 사용자가 "7일 이내 환불"로 질문해도 두 청크 모두 핵심 키워드의 절반씩만 갖고 있어 임베딩 유사도가 낮아진다.
- **오답 위험**: LLM이 절반만 받은 청크로 답을 만들면 잘못된 정보나 환각이 나오기 쉽다.

오버랩이 있으면

- **맥락 보존:** 끊긴 문장이 다음 청크의 앞부분에 통째로 다시 등장하므로, 둘 중 하나는 반드시 온전한 문장을 포함한다.
- **검색 적중률 향상:** 핵심 키워드가 한 청크 안에 모여 있으므로 임베딩이 질문 의도를 정확히 잡아낸다.

한 문장의 길이가 너무 길거나 오버랩 사이즈가 너무 작으면 맥락이 보존되지 않을 수 있으므로, 보통 `chunk_size` 의 10~20% 수준(예: 300자 청크에 50자)에서 시작해 데이터 특성에 맞게 조정하는 것을 권장한다.

문단 기반 청킹

`\n\n`으로 자르되 너무 긴 문단은 추가로 잘라준다. 의미 단위가 잘 보존된다.

- 글을 쓸 때 사람은 하나의 주제·논점이 끝났을 때만 빈 줄(`\n\n`)을 넣는 경우가 대부분이다.
- 고정 길이 청킹은 빈 줄을 무시하고 글자 수만 보고 자르기 때문에 한 주제가 두 청크로 쪼개질 수 있다.
- 문단 기반은 작성자가 그어둔 경계를 그대로 따라가므로 **한 청크 = 한 주제**가 된다.
- `\n\n`은 문장이 끝난 다음에 나타나므로 문단 경계에서 자르면 문장이 절단될 가능성이 없다.

청킹 결과 비교 — 같은 질문, 다른 결과

입력 질문: "환불 정책은 어떻게 되나요?"

같은 PDF, 같은 질문이라도 청킹 전략에 따라 가져오는 청크가 달라지고 답변 품질도 함께 바뀐다.

- 첫째, 오버랩이 있는 청킹은 답이 청크 경계에 걸쳐 있어도 안정적으로 찾는다.
- 둘째, 문단 청킹은 짧은 문단을 잘 보존하지만 긴 문단에서는 정확도가 떨어질 수 있다.